

Apache Kafka Connect Lab Guide

Move data between files, PostgreSQL and Kafka with free connectors only - FileStream and JDBC - then transform it with SMTs, catch bad records in a dead letter queue, and scale the Connect cluster. Every output was captured on the CLI lab's three-broker Kafka 4.3.1 cluster.

FILESTREAM

JDBC

POSTGRESQL

SMT

PREDICATES

DLQ

REST API

WORKERS

0 Overview and Setup

1 The Connect Worker

2 File Source: from a File into a Topic

3 File Sink: from a Topic into a File

4 Single Message Transforms

5 Database to Database with JDBC

6 Running Connect: Errors, Pausing, Scaling, Replaying

7 Cheat Sheet and Troubleshooting

0

1

2

3

4

5

6

7

8

9

10

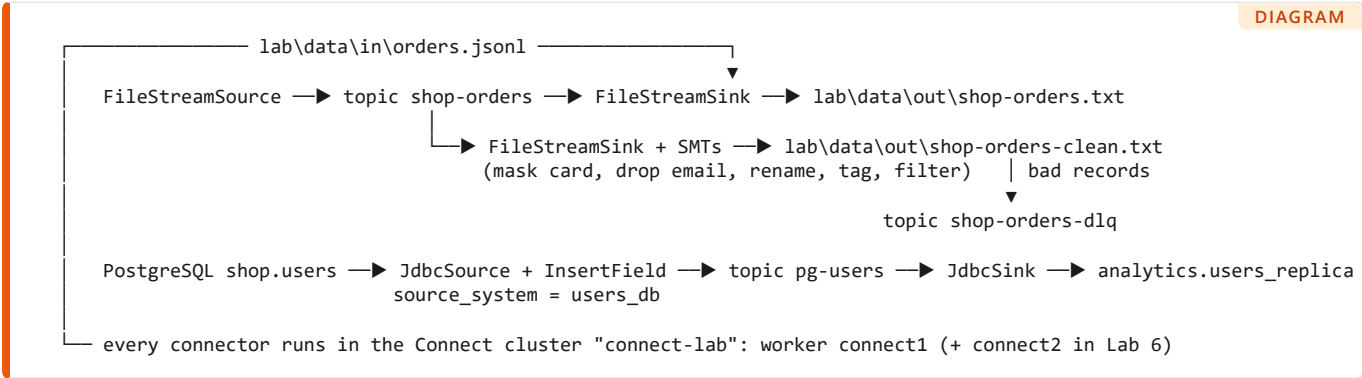
11

Lab 0 · Overview and Setup

Goal: see what you will build, start a Kafka Connect worker and a PostgreSQL database next to the CLI lab's cluster, and check that Connect answers.

What you will build

Kafka Connect moves data between Kafka and other systems using **connectors you configure, not code you write**. In this lab you run Connect from the same `apache/kafka:4.3.1` image as the brokers and build three pipelines, using only free connectors:



Connector	Ships with	License	Used in
<code>FileStreamSourceConnector</code>	Apache Kafka (<code>libs/connect-file-4.3.1.jar</code>)	Apache 2.0	Labs 2, 4, 6
<code>FileStreamSinkConnector</code>	Apache Kafka	Apache 2.0	Labs 3, 4, 6
26 SMTs and 3 predicates	Apache Kafka (<code>connect-transforms</code>)	Apache 2.0	Labs 4, 5, 6
<code>JdbcSourceConnector</code> , <code>JdbcSinkConnector</code>	Confluent Hub, <code>kafka-connect-jdbc</code> 10.9.9	Confluent Community License - free to use	Labs 5, 6

THE FILESTREAM CONNECTORS ARE FOR LEARNING

The Kafka project calls the FileStream connectors examples. They read and write one local file, they cannot run more than one task usefully, and they do not survive a file being rotated. Use them to learn how Connect works, not to ship logs to production.

What you need

What	Check
The Kafka CLI Lab cluster, running	In <code>kafka_cli_lab</code> : <code>docker compose ps</code> shows <code>kafka1</code> , <code>kafka2</code> , <code>kafka3</code> (healthy)
Docker Desktop with about 1.5 GB of free memory	Connect needs 512 MB per worker, PostgreSQL about 100 MB
Internet access, once	Lab 5 downloads the JDBC connector from Confluent Hub (28 MB unpacked)
<code>curl.exe</code>	Ships with Windows 10 and 11

The lab joins the CLI lab's Docker network, `kafka_cli_lab_default`. The Connect workers reach the brokers as `kafka1:9092` and so on, and the brokers see them as ordinary clients.

The files

DIAGRAM

```
kafka_connect_lab/
├── docker-compose.yml          connect1, connect2 (profile "scale"), postgres
├── get-jdbc-connector.ps1      downloads the JDBC connector into plugins\ (Lab 5)
├── Kafka_Connect_Lab_Guide.pdf this book
├── lab/                        mounted at /lab in the Connect containers
│   ├── config/connect1.properties worker settings (connect2.properties: the same, other host name)
│   ├── connectors/*.json       one file per connector configuration
│   ├── data/in/orders.jsonl    the file the source connector reads
│   ├── data/out/               where the sink connectors write
│   ├── sql/01_shop.sql         creates and fills shop.users, creates database analytics
│   └── add-order.sh             appends an order to orders.jsonl (from inside the container)
├── plugins/                    mounted at /plugins - extra connectors go here
├── guide/, build_pdf.py        the chapters of this book and the script that builds it
└── _capture/out/               the raw outputs captured for this book
```

Exercise 0.1 · Start Connect and PostgreSQL

Task: with the CLI lab's cluster running, start this lab's containers and wait until `connect1` is healthy.

✓ SOLUTION

POWERSHELL

```
cd D:\trainings\Apache_Kafka_Training\labs\kafka_connect_lab
docker compose up -d
docker compose ps
```

OUTPUT

NAME	IMAGE	COMMAND	SERVICE	STATUS	PORTS
connect1	apache/kafka:4.3.1	"/opt/kafka/bin/conn..."	connect1	Up 19 seconds (healthy)	0.0.0.0:8083->8083/tcp
postgres	postgres:16-alpine	"docker-entrypoint.s..."	postgres	Up 20 seconds	0.0.0.0:15432->5432/tcp

(Columns trimmed.) The `COMMAND` shows the trick: the broker image, started with `connect-distributed.sh` instead of the broker. A Connect worker is a Kafka **client** that ships in the Kafka distribution, the same way the CLI tools do.

PostgreSQL publishes **15432** on Windows, because 5432 is often taken by a local installation. The lab itself only uses `docker exec`.

Exercise 0.2 · Hello, Connect

Task: call the root of the Connect REST API. Which Kafka cluster is the worker connected to?

✓ SOLUTION

POWERSHELL

```
curl.exe -s localhost:8083/
```

OUTPUT

```
{"version":"4.3.1","commit":"26b251a451ce941d","kafka_cluster_id":"mXRdmJ0FRR-s1q-6q0Gqew"}
```

`kafka_cluster_id` is the `CLUSTER_ID` from the CLI lab's `docker-compose.yml`, so the worker is talking to the right cluster. **Everything in Connect is done through this REST API:** creating connectors, checking their status, pausing them. There is no Connect CLI tool.

SHELLS USED IN THIS BOOK

POWERSHELL blocks run on Windows, in the `kafka_connect_lab` folder. **KAFKA1 SHELL** blocks run in the broker container (`docker exec -it kafka1 bash`). **CONNECT1 SHELL** blocks run in the Connect container (`docker exec -it connect1 bash`). **PSQL** blocks are SQL for `docker exec -it postgres psql -U lab -d shop` .

Lab 1 · The Connect Worker

Goal: read the worker's configuration, see which plugins it can load, and find the internal topics and group in which a Connect cluster keeps its state.

The worker's configuration

connect1.properties

PROPERTIES

```
# -----
# connect1.properties - Kafka Connect worker 1 (distributed mode)
# -----

# the cluster - internal listener names, Connect runs on the Docker network
bootstrap.servers=kafka1:9092,kafka2:9092,kafka3:9092

# workers with the same group.id form one Connect cluster
group.id=connect-lab

# where the Connect cluster keeps its state - three compacted internal topics
config.storage.topic=connect-configs
offset.storage.topic=connect-offsets
status.storage.topic=connect-status
config.storage.replication.factor=3
offset.storage.replication.factor=3
status.storage.replication.factor=3

# default converters - a connector can override them
key.converter=org.apache.kafka.connect.storage.StringConverter
value.converter=org.apache.kafka.connect.json.JsonConverter
value.converter.schemas.enable=false

# save source offsets every 10 s (default 60 s) - quicker to watch in a lab
offset.flush.interval.ms=10000

# Lab only: when a worker leaves, wait at most 30 s (default 5 min) before its
# connectors and tasks are given to the other workers
scheduled.rebalance.max.delay.ms=30000

# REST API
listeners=http://0.0.0.0:8083
rest.advertised.host.name=connect1
rest.advertised.port=8083

# where connector plugins are: the FileStream connectors ship in Kafka's libs,
# anything else (the JDBC connector, Lab 5) goes in /plugins
plugin.path=/opt/kafka/libs/connect-file-4.3.1.jar,/plugins
```

Four groups of settings matter:

- **bootstrap.servers** - Connect is a Kafka client like any other.
- **group.id** - every worker started with **connect-lab** joins the **same Connect cluster**. The workers share the connectors among themselves, much as consumers in a group share partitions. Lab 6 starts a second worker.
- **The three storage topics** - a Connect cluster keeps no state on disk. Connector configs, source offsets and connector status all live **in Kafka**.
- **Converters** - how a connector's data becomes the bytes of a Kafka message, and back. These are defaults: each connector can override them, and most connectors in this lab do.

Two settings only shorten waits for the lab: **offset.flush.interval.ms** (Lab 2) and **scheduled.rebalance.max.delay.ms** (Lab 6).

Exercise 1.1 · What can this worker run?

Task: list the connector plugins the worker found on its `plugin.path`.

✓ SOLUTION

POWERSHELL

```
curl.exe -s localhost:8083/connector-plugins
```

OUTPUT

```
[
  {"class": "org.apache.kafka.connect.file.FileStreamSinkConnector", "type": "sink", "version": "4.3.1"},
  {"class": "org.apache.kafka.connect.file.FileStreamSourceConnector", "type": "source", "version": "4.3.1"},
  {"class": "org.apache.kafka.connect.mirror.MirrorCheckpointConnector", "type": "source", "version": "4.3.1"},
  {"class": "org.apache.kafka.connect.mirror.MirrorHeartbeatConnector", "type": "source", "version": "4.3.1"},
  {"class": "org.apache.kafka.connect.mirror.MirrorSourceConnector", "type": "source", "version": "4.3.1"}
]
```

(One plugin per line, for reading.) Five connectors ship with Apache Kafka. The two **FileStream** connectors are the ones used in this lab. The three **Mirror** connectors are MirrorMaker 2, which copies topics between clusters. A worker can only run a connector whose class it can load. In Lab 5 you add the JDBC connector to this list.

Exercise 1.2 · Transforms, predicates and converters

Task: the same endpoint lists every plugin type when you add `?connectorsOnly=false`. How many transformations ship with Kafka?

✓ SOLUTION

POWERSHELL

```
curl.exe -s "localhost:8083/connector-plugins?connectorsOnly=false"
```

Counted by type, with the transformations and predicates listed:

OUTPUT

```
transformation: 26  header_converter: 10  converter: 9  source: 4  predicate: 3  sink: 1

Cast$Key Cast$Value DropHeaders ExtractField$Key ExtractField$Value Filter
Flatten$Key Flatten$Value HeaderFrom$Key HeaderFrom$Value HoistField$Key HoistField$Value
InsertField$Key InsertField$Value InsertHeader MaskField$Key MaskField$Value RegexRouter
ReplaceField$Key ReplaceField$Value SetSchemaMetadata$Key SetSchemaMetadata$Value
TimestampConverter$Key TimestampConverter$Value TimestampRouter ValueToKey

predicates: HasHeaderKey RecordIsTombstone TopicNameMatches
```

(Package names `org.apache.kafka.connect.transforms[.predicates]` dropped.) These are the **Single Message Transforms** from the lesson. Most come in a `$Key` and a `$Value` flavour, depending on which part of the record they change. You will use `MaskField`, `ReplaceField`, `InsertField`, `Filter`, `HasHeaderKey`, `ValueToKey` and `ExtractField`.

Exercise 1.3 · Where Connect keeps its state

Task: find the worker's internal topics, and describe them. What kind of topics are they?

✓ SOLUTION

KAFKA1 SHELL

```
kafka-topics.sh --bootstrap-server kafka1:9092 --list | grep connect
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic connect-offsets | head -1
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic connect-configs | head -1
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic connect-status | head -1
```

OUTPUT

```
connect-configs
connect-offsets
connect-status
Topic: connect-offsets TopicId: WPKG-enmQM2ISPVkvQMqyw PartitionCount: 25 ReplicationFactor: 3 Configs:
min.insync.replicas=2,cleanup.policy=compact
Topic: connect-configs TopicId: y_v_b1SOTwqNAqG_x1XVvg PartitionCount: 1 ReplicationFactor: 3 Configs:
min.insync.replicas=2,cleanup.policy=compact
Topic: connect-status TopicId: PvKJnb7wSBSuZOR2C6gy1w PartitionCount: 5 ReplicationFactor: 3 Configs:
min.insync.replicas=2,cleanup.policy=compact
```

The worker created all three on its first start. They are **compacted** topics (CLI Lab 4): only the latest value per key matters, which is exactly what you want for "the config of connector X" or "the offset of file Y". `connect-configs` has a **single partition** on purpose, so that every worker reads the config changes in the same order.

This is why you can stop every worker, start them again, and find all your connectors running as before.

Exercise 1.4 · The worker group

Task: list every group in the cluster with `kafka-groups.sh`. Is `connect-lab` a consumer group?

✓ SOLUTION

KAFKA1 SHELL

```
kafka-groups.sh --bootstrap-server kafka1:9092 --list
```

OUTPUT

GROUP	TYPE	PROTOCOL
connect-lab	Classic	connect
orders-api	Classic	consumer

`connect-lab` uses the group coordinator from the Consumers lesson, but with protocol `connect` instead of `consumer`. The workers use it to share connectors and tasks, not partitions. `kafka-consumer-groups.sh` ignores such groups, which is why `kafka-groups.sh` exists. (`orders-api` is from the Spring Boot lab. Your list may have other groups.)

Lab 2 · File Source: from a File into a Topic

Goal: create your first connector with one REST call. Watch it read a file into a topic and follow new lines as they are added. Then see how it remembers its place, and what a converter does.

The input is `lab\data\in\orders.jsonl`: one order per line, as JSON, with personal data such as an email and a card number. In Lab 4 you mask those.

OUTPUT

```
{
  "order": 3001, "customer": "alice", "item": "keyboard", "amount": 45, "email": "alice@example.com", "card": "4111111111111111"
}
{
  "order": 3002, "customer": "dave", "item": "monitor", "amount": 210, "email": "dave@example.com", "card": "5500005555555559"
}
... six orders, 3001 to 3006
```

Exercise 2.1 · Create the connector

Task: create the connector `orders-file-source` from `lab\connectors\orders-file-source.json`.

`orders-file-source.json`

JSON

```
{
  "connector.class": "org.apache.kafka.connect.file.FileStreamSourceConnector",
  "tasks.max": "1",
  "file": "/lab/data/in/orders.jsonl",
  "topic": "shop-orders",
  "value.converter": "org.apache.kafka.connect.storage.StringConverter",
  "topic.creation.default.partitions": "3",
  "topic.creation.default.replication.factor": "3"
}
```

✓ SOLUTION

POWERSHELL

```
curl.exe -s -i -X PUT localhost:8083/connectors/orders-file-source/config -H "Content-Type: application/json" --data "@lab/connectors/orders-file-source.json"
```

OUTPUT

```
HTTP/1.1 201 Created
Location: http://localhost:8083/connectors/orders-file-source
Content-Type: application/json

{"name": "orders-file-source", "config": {
  "connector.class": "org.apache.kafka.connect.file.FileStreamSourceConnector", "tasks.max": "1", "file": "/lab/data/in/orders.jsonl", "topic": "shop-orders", ... }, "tasks": [], "type": "source"}
```

That is the whole deployment: **a name and a JSON config, sent with `PUT /connectors/<name>/config`. `201 Created`** means the connector is new. Send the same request again and you get `200 OK`, which is why `PUT` is the verb to use in scripts: it creates or updates, and repeating it does no harm.

What each line of the config does:

Setting	Meaning
<code>connector.class</code>	The plugin to run - it must be on the worker's <code>plugin.path</code>
<code>tasks.max</code>	At most this many tasks. A file can only be read by one
<code>file</code> , <code>topic</code>	The connector's own settings - every connector has different ones

Setting	Meaning
<code>value.converter</code>	Overrides the worker default: write each line as plain text, not as JSON (Exercise 2.5)
<code>topic.creation.default.*</code>	Let Connect create <code>shop-orders</code> with 3 partitions and replication factor 3. The cluster has <code>auto.create.topics.enable=false</code>

Exercise 2.2 · Is it running?

Task: check the connector's status, then look at the topic it created and read it.

✓ SOLUTION

```
curl.exe -s localhost:8083/connectors/orders-file-source/status
```

POWERSHELL

OUTPUT

```
{
  "name": "orders-file-source",
  "connector": {"state": "RUNNING", "worker_id": "connect1:8083", "version": "4.3.1"},
  "tasks": [
    {"id": 0, "state": "RUNNING", "worker_id": "connect1:8083", "version": "4.3.1"}
  ],
  "type": "source"
}
```

A **connector** plans the work, and its **tasks** do it. Both are running on worker `connect1`.

```
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic shop-orders | head -1
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic shop-orders --from-beginning --max-
messages 6 \
  --formatter-property print.key=true --formatter-property print.partition=true --formatter-property
print.offset=true
```

KAFKA1 SHELL

OUTPUT

```
Topic: shop-orders TopicId: vUUE-VrSQ6SSkMx9EiWGGA PartitionCount: 3 ReplicationFactor: 3 Configs:
min.insync.replicas=2
Partition:2 Offset:0 null
{"order":3001,"customer":"alice","item":"keyboard","amount":45,"email":"alice@example.com","card":"4111111111111111"}
Partition:2 Offset:1 null
{"order":3002,"customer":"dave","item":"monitor","amount":210,"email":"dave@example.com","card":"5500005555555559"}
Partition:2 Offset:2 null
{"order":3003,"customer":"carol","item":"mouse","amount":18,"email":"carol@example.com","card":"3400000000000009"}
Partition:2 Offset:3 null
{"order":3004,"customer":"alice","item":"headset","amount":60,"email":"alice@example.com","card":"4111111111111111"}
Partition:2 Offset:4 null
{"order":3005,"customer":"erin","item":"webcam","amount":75,"email":"erin@example.com","card":"6011000000000004"}
Partition:2 Offset:5 null
{"order":3006,"customer":"frank","item":"dock","amount":140,"email":"frank@example.com","card":"5105105105105100"}
Processed a total of 6 messages
```

One message per line, all with key `null`: a line of a file has no key. With no key, the producer inside Connect used the **sticky partitioner** (CLI Lab 5), so all six lines went into one batch and one partition, partition 2.

Exercise 2.3 · New lines arrive

Task: add two orders to the end of `orders.jsonl`. Try it from PowerShell first.

✓ SOLUTION

POWERSHELL

```
Add-Content lab\data\in\orders.jsonl '{"order":3007,"customer":"bob","item":"cable","amount":9}'
```

OUTPUT

```
Add-Content : The process cannot access the file
'D:\trainings\Apache_Kafka_Training\labs\kafka_connect_lab\lab\data\in\orders.jsonl'
because it is being used by another process.
```

The connector keeps the file **open** so it can follow it, and Docker Desktop then refuses Windows programs write access to it. Write from **inside** the container instead, where the file is an ordinary Linux file:

POWERSHELL

```
docker exec -it connect1 bash
```

CONNECT1 SHELL

```
echo
'{"order":3007,"customer":"bob","item":"cable","amount":9,"email":"bob@example.com","card":"4012888888888881"}' >> /lab/data/in/orders.jsonl
echo
'{"order":3008,"customer":"dave","item":"chair","amount":320,"email":"dave@example.com","card":"5500005555555559"}' >> /lab/data/in/orders.jsonl
exit
```

KAFKA1 SHELL

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic shop-orders --partition 2 --offset 6 --
max-messages 2 \
--formatter-property print.offset=true
```

OUTPUT

```
Offset:6
{"order":3007,"customer":"bob","item":"cable","amount":9,"email":"bob@example.com","card":"4012888888888881"}
Offset:7
{"order":3008,"customer":"dave","item":"chair","amount":320,"email":"dave@example.com","card":"5500005555555559"}
```

Both lines were in Kafka within a second. The source task polls the file for new lines, much as `tail -f` does.

A SHORTCUT FOR THE REST OF THE LAB

`lab\add-order.sh` appends one order for you, and runs inside the container:

POWERSHELL

```
docker exec connect1 bash /lab/add-order.sh 3009 carol "desk lamp" 35
```

Exercise 2.4 · How does it know where it was?

Task: ask Connect for the connector's **offsets**, and compare them with the size of the file. Then restart the connector and its task. Are the orders sent again?

✓ SOLUTION

POWERSHELL

```
curl.exe -s localhost:8083/connectors/orders-file-source/offsets
(Get-Item lab\data\in\orders.jsonl).Length
```

OUTPUT

```
{"offsets":[{"partition":{"filename":"/lab/data/in/orders.json"},"offset":{"position":918}}]}
```

A source connector's offset is not a Kafka offset but a **position in the source system**, in whatever form suits that system. For a file, it is the byte position: 918, the file's length, so the connector has read everything. The worker stores it in `connect-offsets`, every `offset.flush.interval.ms` (10 s in this lab).

POWERSHELL

```
curl.exe -s -i -X POST "localhost:8083/connectors/orders-file-source/restart?includeTasks=true"
```

KAFKA1 SHELL

```
kafka-get-offsets.sh --bootstrap-server kafka1:9092 --topic shop-orders
```

OUTPUT

```
HTTP/1.1 202 Accepted
shop-orders:0:0
shop-orders:1:0
shop-orders:2:8
```

Still 8 messages. The restarted task read its stored position and carried on from byte 918, without reading the file again. (A crash **between** writing to Kafka and saving the offset can repeat the last few lines. Source connectors are at-least-once unless they support exactly-once.)

Exercise 2.5 · What a converter does

Task: create a second source connector on the same file, without the `value.converter` line, so it uses the worker default `JsonConverter`. Its topic is `shop-orders-json`. Compare one message with `shop-orders`, then delete the connector.

✓ SOLUTION

`lab/connectors/orders-file-source-json.json` is the same config without the converter line, writing to `shop-orders-json` (one partition):

POWERSHELL

```
curl.exe -s -X PUT localhost:8083/connectors/orders-file-source-json/config -H "Content-Type: application/json" --data "@lab/connectors/orders-file-source-json.json"
```

KAFKA1 SHELL

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic shop-orders-json --from-beginning --max-messages 1
```

OUTPUT

```
"
{"order":3001,"customer":"alice","item":"keyboard","amount":45,"email":"alice@example.com","card":"4111111111111111"}"
Processed a total of 1 messages
```

The `FileStream` source hands each line to Connect as a **string**. The `JsonConverter` then did its job: it wrote that string as a **JSON string**, with quotes around it and every inner quote escaped. Nothing failed, and the data is still awkward for any consumer to use. **The connector decides what the data is; the converter decides how it becomes bytes.** For text that is already JSON, `StringConverter` is the right choice.

POWERSHELL

```
curl.exe -s -i -X DELETE localhost:8083/connectors/orders-file-source-json
```

HTTP/1.1 204 No Content

OUTPUT

Deleting a connector stops it and removes its config. Its topic, `shop-orders-json`, stays. You can delete it with `kafka-topics.sh --delete` if you like.

Lab 3 · File Sink: from a Topic into a File

Goal: add the other half: a sink connector that writes `shop-orders` to a file. You then have a complete pipeline, file → Kafka → file. You will also see that a sink connector is, underneath, a consumer group.

Exercise 3.1 · Create the sink

Task: create `orders-file-sink` from `lab\connectors\orders-file-sink.json`, check its status, and open the file it writes.

`orders-file-sink.json`

JSON

```
{
  "connector.class": "org.apache.kafka.connect.file.FileStreamSinkConnector",
  "tasks.max": "1",
  "topics": "shop-orders",
  "file": "/lab/data/out/shop-orders.txt",
  "value.converter": "org.apache.kafka.connect.storage.StringConverter"
}
```

✓ SOLUTION

POWERSHELL

```
curl.exe -s -X PUT localhost:8083/connectors/orders-file-sink/config -H "Content-Type: application/json"
--data "@lab/connectors/orders-file-sink.json"
curl.exe -s localhost:8083/connectors/orders-file-sink/status
```

OUTPUT

```
{
  "name": "orders-file-sink",
  "config": {
    "connector.class": "org.apache.kafka.connect.file.FileStreamSinkConnector",
    "tasks": [],
    "type": "sink"
  },
  "name": "orders-file-sink",
  "connector": {
    "state": "RUNNING",
    "worker_id": "connect1:8083",
    "version": "4.3.1",
    "tasks": [
      {
        "id": 0,
        "state": "RUNNING",
        "worker_id": "connect1:8083",
        "version": "4.3.1",
        "type": "sink"
      }
    ]
  }
}
```

POWERSHELL

```
Get-Content lab\data\out\shop-orders.txt
```

OUTPUT

```
{
  "order": 3001,
  "customer": "alice",
  "item": "keyboard",
  "amount": 45,
  "email": "alice@example.com",
  "card": "4111111111111111"
}, {
  "order": 3002,
  "customer": "dave",
  "item": "monitor",
  "amount": 210,
  "email": "dave@example.com",
  "card": "5500005555555559"
}, {
  "order": 3003,
  "customer": "carol",
  "item": "mouse",
  "amount": 18,
  "email": "carol@example.com",
  "card": "3400000000000009"
}, {
  "order": 3004,
  "customer": "alice",
  "item": "headset",
  "amount": 60,
  "email": "alice@example.com",
  "card": "4111111111111111"
}, {
  "order": 3005,
  "customer": "erin",
  "item": "webcam",
  "amount": 75,
  "email": "erin@example.com",
  "card": "6011000000000004"
}, {
  "order": 3006,
  "customer": "frank",
  "item": "dock",
  "amount": 140,
  "email": "frank@example.com",
  "card": "5105105105105100"
}, {
  "order": 3007,
  "customer": "bob",
  "item": "cable",
  "amount": 9,
  "email": "bob@example.com",
  "card": "4012888888881881"
}, {
  "order": 3008,
  "customer": "dave",
  "item": "chair",
  "amount": 320,
  "email": "dave@example.com",
  "card": "5500005555555559"
}
```

A sink uses `topics`, plural (or `topics.regex`): **one sink can read many topics**. It read `shop-orders` from the beginning, so the file holds all eight orders. Windows can **read** the output file while the connector has it open. Only writing to it is blocked.

Exercise 3.2 · A sink is a consumer group

Task: a sink connector consumes, so it must belong to a consumer group. Find it with `kafka-consumer-groups.sh`.

✓ SOLUTION

KAFKA1 SHELL

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group connect-orders-file-sink
```

OUTPUT

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG	CONSUMER-ID
connect-orders-file-sink-0-5e7f632d	shop-orders	0	0	0	0	connector-consumer-orders-file-
connect-orders-file-sink-0-5e7f632d	shop-orders	1	0	0	0	connector-consumer-orders-file-
connect-orders-file-sink-0-5e7f632d	shop-orders	2	8	8	0	connector-consumer-orders-file-

(Spacing tightened; `HOST` and `CLIENT-ID` columns and the end of each consumer id left out.) Every sink connector gets the group `connect-<connector name>`, and each task is a member of it. Unlike a source connector's file position, **a sink's offsets are ordinary committed consumer offsets** in `__consumer_offsets`. Everything from the Consumers lesson applies: lag, `--reset-offsets` while the connector is stopped, and at-least-once delivery.

Exercise 3.3 · The whole pipeline, live

Task: add order 3009 with `add-order.sh` and look at the last lines of the output file.

✓ SOLUTION

POWERSHELL

```
docker exec connect1 bash /lab/add-order.sh 3009 carol "desk lamp" 35
Get-Content lab\data\out\shop-orders.txt -Tail 2
```

OUTPUT

```
{"order":3009,"customer":"carol","item":"desk lamp","amount":35,"email":"carol@example.com","card":"4111111111111111"}
{"order":3008,"customer":"dave","item":"chair","amount":320,"email":"dave@example.com","card":"5500005555555559"}
{"order":3009,"customer":"carol","item":"desk lamp","amount":35,"email":"carol@example.com","card":"4111111111111111"}
```

One line appended to one file showed up in another file, about a second later. Between them: a source connector producing, a topic replicated three times, and a sink connector consuming. No code was written. Any other consumer can read the same orders from `shop-orders` at the same time, or a week later.

Exercise 3.4 · All connectors at once

Task: list every connector with its status in one call.

✓ SOLUTION

POWERSHELL

```
curl.exe -s "localhost:8083/connectors?expand=status"
```

One line per connector, shortened from the JSON:

OUTPUT

```
orders-file-sink    sink    RUNNING  task 0 RUNNING on connect1:8083
orders-file-source  source  RUNNING  task 0 RUNNING on connect1:8083
```

`?expand=status` (or `?expand=info` for the configs) turns the list of names from `GET /connectors` into a full overview. It is the call to use in a monitoring script.

Lab 4 · Single Message Transforms

Goal: clean the orders on their way out of Kafka with a chain of built-in SMTs: mask the card number, drop the email, rename a field, and tag each message with where it came from. Then drop test messages with a `Filter` and a predicate. You write no code, only configuration.

Exercise 4.1 · A chain of transforms

Task: create `orders-clean-sink`. It reads `shop-orders` again, parses each message as JSON, transforms it, and writes it to `lab\data\out\shop-orders-clean.txt`. Read the config first: what will a line look like?

`orders-clean-sink.json`

JSON

```
{
  "connector.class": "org.apache.kafka.connect.file.FileStreamSinkConnector",
  "tasks.max": "1",
  "topics": "shop-orders",
  "file": "/lab/data/out/shop-orders-clean.txt",
  "value.converter": "org.apache.kafka.connect.json.JsonConverter",
  "value.converter.schemas.enable": "false",

  "transforms": "mask, rename, tag",

  "transforms.mask.type": "org.apache.kafka.connect.transforms.MaskField$Value",
  "transforms.mask.fields": "card",
  "transforms.mask.replacement": "****",

  "transforms.rename.type": "org.apache.kafka.connect.transforms.ReplaceField$Value",
  "transforms.rename.exclude": "email",
  "transforms.rename.renames": "amount:total",

  "transforms.tag.type": "org.apache.kafka.connect.transforms.InsertField$Value",
  "transforms.tag.static.field": "source_system",
  "transforms.tag.static.value": "orders_feed",
  "transforms.tag.partition.field": "kafka_partition",
  "transforms.tag.offset.field": "kafka_offset"
}
```

✓ SOLUTION

POWERSHELL

```
curl.exe -s -o NUL -w "HTTP %{http_code}\n" -X PUT localhost:8083/connectors/orders-clean-sink/config -H
"Content-Type: application/json" --data "@lab/connectors/orders-clean-sink.json"
Get-Content lab\data\out\shop-orders-clean.txt
```

OUTPUT

```
HTTP 201
{item=keyboard, total=45, kafka_offset=0, kafka_partition=2, source_system=orders_feed, card=****, order=3001,
customer=alice}
{item=monitor, total=210, kafka_offset=1, kafka_partition=2, source_system=orders_feed, card=****, order=3002,
customer=dave}
{item=mouse, total=18, kafka_offset=2, kafka_partition=2, source_system=orders_feed, card=****, order=3003,
customer=carol}
{item=headset, total=60, kafka_offset=3, kafka_partition=2, source_system=orders_feed, card=****, order=3004,
customer=alice}
{item=webcam, total=75, kafka_offset=4, kafka_partition=2, source_system=orders_feed, card=****, order=3005,
customer=erin}
{item=dock, total=140, kafka_offset=5, kafka_partition=2, source_system=orders_feed, card=****, order=3006,
customer=frank}
{item=cable, total=9, kafka_offset=6, kafka_partition=2, source_system=orders_feed, card=****, order=3007,
customer=bob}
{item=chair, total=320, kafka_offset=7, kafka_partition=2, source_system=orders_feed, card=****, order=3008,
customer=dave}
```

```
{item=desk lamp, total=35, kafka_offset=0, kafka_partition=0, source_system=orders_feed, card=****, order=3009, customer=carol}
```

What happened to every record, step by step:

Step	What it did to <code>{"order":3001, ..., "amount":45, "email":"alice@...", "card":"4111..."}</code>
<code>JsonConverter</code> (<code>schemas.enable=false</code>)	Parsed the bytes into a map of fields. SMTs need structure : they cannot change a field inside a plain string
<code>mask</code> - <code>MaskField\$Value</code>	<code>card</code> became <code>****</code>
<code>rename</code> - <code>ReplaceField\$Value</code>	<code>email</code> removed, <code>amount</code> renamed to <code>total</code>
<code>tag</code> - <code>InsertField\$Value</code>	Added <code>source_system=orders_feed</code> (a fixed value), plus <code>kafka_partition</code> and <code>kafka_offset</code> (where the record came from)
<code>FileStreamSink</code>	Wrote the result as text

The transforms run **in the order listed in** `transforms`, each one on the previous one's output. The `{key=value, ...}` format is how a Java map prints itself, which is all the `FileStream` sink knows how to do. A real sink, such as the `JDBC` sink in Lab 5, writes proper columns.

Order 3009 is in partition **0**: the source's producer had moved on to a new batch, and so to another partition. `shop-orders` itself is unchanged. The SMTs only shaped what this one sink wrote. `orders-file-sink` and any consumer still see the full data, card numbers included.

MASKING ON THE WAY OUT IS TOO LATE FOR REAL PII

Here the card numbers are masked only in one output file. They are still stored in the topic, in clear text. To keep personal data out of Kafka completely, mask it in the **source** connector, before it is ever written. Lab 5 puts SMTs on a source.

Exercise 4.2 · Filter out test messages

Task: a QA tool marks its test orders with a header named `test`. Change `orders-clean-sink` so that it drops every message that has this header, and only those. Then send one test order and one real order.

✓ SOLUTION

`Filter` drops every record, so on its own it is useless. Combined with a **predicate**, it drops only the records the predicate matches. `lab\connectors\orders-clean-sink-filter.json` puts it first in the chain:

```
"transforms": "dropTests,mask,rename,tag",

"transforms.dropTests.type": "org.apache.kafka.connect.transforms.Filter",
"transforms.dropTests.predicate": "isTest",
"predicates": "isTest",
"predicates.isTest.type": "org.apache.kafka.connect.transforms.predicates.HasHeaderKey",
"predicates.isTest.name": "test",
```

PUT the new config to the **same** connector name. That updates it, and the task restarts with the new chain:

```
curl.exe -s -o NUL -w "HTTP %{http_code}\n" -X PUT localhost:8083/connectors/orders-clean-sink/config -H
```

```
"Content-Type: application/json" --data "@lab/connectors/orders-clean-sink-filter.json"
```

HTTP 200

OUTPUT

Send a test order with a header, then a real one. The console producer reads a header from the start of the line, up to a tab:

```
kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic shop-orders --reader-property
parse.headers=true
>test:yes {"order":9999,"customer":"qa-bot","item":"test
item","amount":1,"email":"qa@example.com","card":"0000000000000000"}
>^C
```

KAFKA1 SHELL

```
docker exec connect1 bash /lab/add-order.sh 3010 erin "usb hub" 22
Get-Content lab\data\out\shop-orders.txt -Tail 2
Get-Content lab\data\out\shop-orders-clean.txt -Tail 2
```

POWERSHELL

```
{"order":9999,"customer":"qa-bot","item":"test item","amount":1,"email":"qa@example.com","card":"0000000000000000"}
{"order":3010,"customer":"erin","item":"usb hub","amount":22,"email":"erin@example.com","card":"4111111111111111"}

{item=desk lamp, total=35, kafka_offset=0, kafka_partition=0, source_system=orders_feed, card=****, order=3009,
customer=carol}
{item=usb hub, total=22, kafka_offset=2, kafka_partition=0, source_system=orders_feed, card=****, order=3010,
customer=erin}
```

OUTPUT

The plain sink wrote both messages. The clean sink jumped from offset 0 to offset 2: offset 1, the test order, was dropped. Check with the headers shown:

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic shop-orders --partition 0 --offset 0 --
max-messages 3 \
--formatter-property print.headers=true --formatter-property print.offset=true
```

KAFKA1 SHELL

```
Offset:0 NO_HEADERS {"order":3009,"customer":"carol","item":"desk lamp","amount":35,...}
Offset:1 test:yes {"order":9999,"customer":"qa-bot","item":"test item","amount":1,...}
Offset:2 NO_HEADERS {"order":3010,"customer":"erin","item":"usb hub","amount":22,...}
```

OUTPUT

(Values shortened.) Kafka ships three predicates: `HasHeaderKey`, `TopicNameMatches` and `RecordIsTombstone`. They test the record's envelope, not the contents of its fields. To filter on a field value, such as "orders over 100", you need either an SMT from another project or **stream processing** (Module 10).

Exercise 4.3 · Think about it

1. Why can no SMT turn `{"amount":45}` and `{"amount":60}` into a running total for alice? (SMTs see one message at a time and remember nothing. That is the "stateless" rule from the lesson.)
2. You add `ValueToKey` with `fields=customer` to `orders-file-source`. Why does it fail? (The source produces a plain string. Like every SMT that works on fields, `ValueToKey` needs structured data, which a JSON-parsing converter on a sink, or a structured source such as JDBC, provides.)
3. `ReplaceField` used to be configured with `blacklist` and `whitelist`. Which settings replaced them? (`exclude` and `include`.)

Lab 5 · Database to Database with JDBC

Goal: build the lesson's own example. A source connector reads the `users` table from PostgreSQL and adds `source_system = users_db` to every message. A sink connector then writes those messages into a table in **another** database. You will see inserts and updates flow through, and the sink create its table by itself.

The JDBC connector is not part of Apache Kafka. It is Confluent's, free to use under the Confluent Community License, and installed from Confluent Hub. It polls tables with SQL queries, so it needs no special setup in the database. (A CDC connector such as Debezium reads the database's transaction log instead, and so also sees deletes. It is free too, but needs more setup.)

Exercise 5.1 · Install a connector plugin

Task: download the JDBC connector into `plugins\`, restart the worker, and check that it now lists the JDBC connectors.

✓ SOLUTION

```
powershell -ExecutionPolicy Bypass -File .\get-jdbc-connector.ps1
```

POWERSHELL

```
Downloading https://hub-downloads.confluent.io/api/plugins/confluentinc/kafka-connect-jdbc/versions/10.9.9/confluentinc-kafka-connect-jdbc-10.9.9.zip
Installed: plugins\confluentinc-kafka-connect-jdbc-10.9.9
```

OUTPUT

Name	KB
----	--
common-utils-7.0.12.jar	17
kafka-connect-jdbc-10.9.9.jar	359
mssql-jdbc-12.8.2.jre8.jar	1178
ojdbc8-19.7.0.0.jar	4296
postgresql-42.7.12.jar	1116
sqlite-jdbc-3.41.2.2.jar	12550
...	(19 jars)

The connector, `kafka-connect-jdbc`, comes **with the JDBC drivers** it needs: PostgreSQL, SQL Server, Oracle and SQLite. That is a plugin: a folder of jars that the worker loads in its **own class loader**, isolated from other plugins, so two connectors can use different versions of the same library.

```
docker compose restart connect1
curl.exe -s localhost:8083/connector-plugins
```

POWERSHELL

```
[
  {"class": "io.confluent.connect.jdbc.JdbcSinkConnector", "type": "sink", "version": "10.9.9"},
  {"class": "org.apache.kafka.connect.file.FileStreamSinkConnector", "type": "sink", "version": "4.3.1"},
  {"class": "io.confluent.connect.jdbc.JdbcSourceConnector", "type": "source", "version": "10.9.9"},
  {"class": "org.apache.kafka.connect.file.FileStreamSourceConnector", "type": "source", "version": "4.3.1"},
  ... the three Mirror connectors
]
```

OUTPUT

Workers scan `plugin.path` **only at startup**, so a new plugin needs a restart. Notice that your three connectors carried on after the restart without you doing anything. Their configs and offsets are in Kafka (Lab 1).

Exercise 5.2 · The source table

Task: look at the `users` table in the `shop` database.

✓ SOLUTION

```
docker exec -it postgres psql -U lab -d shop
```

POWERSHELL

```
SELECT * FROM users;
```

PSQL

```
id | name | email | country | updated_at
---+---+-----+-----+-----
 1 | ana | ana@example.com | PT | 2026-10-01 01:29:51.579
 2 | raj | raj@example.com | IN | 2026-10-01 01:29:51.579
 3 | li | li@example.com | CN | 2026-10-01 01:29:51.579
 4 | sam | sam@example.com | US | 2026-10-01 01:29:51.579
(4 rows)
```

OUTPUT

`lab\sql\01_shop.sql` created this table when the container first started. Two columns matter to the connector: `id` only ever grows, and a trigger sets `updated_at` on every `UPDATE`.

Exercise 5.3 · The source connector, with the lesson's SMT

Task: create `users-db-source` and read the topic it fills. What is the key of each message? Where did `source_system` come from?

`users-db-source.json`

JSON

```
{
  "connector.class": "io.confluent.connect.jdbc.JdbcSourceConnector",
  "tasks.max": "1",
  "connection.url": "jdbc:postgresql://postgres:5432/shop",
  "connection.user": "lab",
  "connection.password": "lab",
  "table.whitelist": "users",
  "mode": "timestamp+incrementing",
  "incrementing.column.name": "id",
  "timestamp.column.name": "updated_at",
  "poll.interval.ms": "2000",
  "topic.prefix": "pg-",
  "topic.creation.default.partitions": "3",
  "topic.creation.default.replication.factor": "3",

  "value.converter": "org.apache.kafka.connect.json.JsonConverter",
  "value.converter.schemas.enable": "true",

  "transforms": "addSource,idToKey,keyOnly",

  "transforms.addSource.type": "org.apache.kafka.connect.transforms.InsertField$Value",
  "transforms.addSource.static.field": "source_system",
  "transforms.addSource.static.value": "users_db",

  "transforms.idToKey.type": "org.apache.kafka.connect.transforms.ValueToKey",
  "transforms.idToKey.fields": "id",

  "transforms.keyOnly.type": "org.apache.kafka.connect.transforms.ExtractField$Key",
```

```
"transforms.keyOnly.field": "id"
}
```

✓ SOLUTION

POWERSHELL

```
curl.exe -s -o NUL -w "HTTP %{http_code}\n" -X PUT localhost:8083/connectors/users-db-source/config -H
"Content-Type: application/json" --data "@lab/connectors/users-db-source.json"
```

KAFKA1 SHELL

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic pg-users --from-beginning --max-
messages 4 \
--formatter-property print.key=true --formatter-property print.partition=true
```

One message in full, formatted:

OUTPUT

```
Partition:2 2  {"schema": {"type": "struct", "name": "users", "optional": false,
    "fields": [{"field": "id", "type": "int32", "optional": false},
    {"field": "name", "type": "string", "optional": false},
    {"field": "email", "type": "string", "optional": false},
    {"field": "country", "type": "string", "optional": false},
    {"field": "updated_at", "type": "int64", "optional": false,
    "name": "org.apache.kafka.connect.data.Timestamp", "version": 1},
    {"field": "source_system", "type": "string", "optional": true}]}},
    "payload": {"id": 2, "name": "raj", "email": "raj@example.com", "country": "IN",
    "updated_at": 1790818191579, "source_system": "users_db"}}
```

The four messages, payload only:

OUTPUT

```
Partition:2 key=2
{"id":2,"name":"raj","email":"raj@example.com","country":"IN","updated_at":1790818191579,"source_system":"users_db"}
Partition:2 key=3
{"id":3,"name":"li","email":"li@example.com","country":"CN","updated_at":1790818191579,"source_system":"users_db"}
Partition:0 key=1
{"id":1,"name":"ana","email":"ana@example.com","country":"PT","updated_at":1790818191579,"source_system":"users_db"}
Partition:1 key=4
{"id":4,"name":"sam","email":"sam@example.com","country":"US","updated_at":1790818191579,"source_system":"users_db"}
```

- **One row, one message**, in topic `topic.prefix` + table name: `pg-users`.
- `source_system: "users_db"` is in every message, though the table has no such column. That is `InsertField`, exactly as in the lesson.
- **The key is the row's `id`**. `ValueToKey` copied `id` into the key as a small struct, and `ExtractField$Key` reduced that struct to the plain value. Now every version of row 3 goes to the same partition, in order.
- **The schema travels with every message**, because `schemas.enable=true`. The JDBC **sink** needs it to know the column types. That costs a lot of bytes per message. With **Schema Registry** (Module 8) and Avro, a message carries a 4-byte schema id instead.

Exercise 5.4 · The sink connector

Task: create `users-db-sink`, which writes `pg-users` into the table `users_replica` in the `analytics` database. The table does not exist yet. Look at it afterwards.

`users-db-sink.json`

JSON

```
{
  "connector.class": "io.confluent.connect.jdbc.JdbcSinkConnector",
  "tasks.max": "1",
  "topics": "pg-users",
```

```

"connection.url": "jdbc:postgresql://postgres:5432/analytics",
"connection.user": "lab",
"connection.password": "lab",
"table.name.format": "users_replica",
"auto.create": "true",
"insert.mode": "upsert",
"pk.mode": "record_value",
"pk.fields": "id",

"value.converter": "org.apache.kafka.connect.json.JsonConverter",
"value.converter.schemas.enable": "true"
}

```

✓ SOLUTION

POWERSHELL

```

curl.exe -s -o NUL -w "HTTP %{http_code}\n" -X PUT localhost:8083/connectors/users-db-sink/config -H
"Content-Type: application/json" --data "@lab/connectors/users-db-sink.json"
docker exec -it postgres psql -U lab -d analytics

```

PSQL

```

\d users_replica
SELECT * FROM users_replica ORDER BY id;

```

OUTPUT

```

      Table "public.users_replica"
  Column      |      Type       | Collation | Nullable | Default
-----+-----+-----+-----+-----
 id           | integer         |           | not null |
 name        | text            |           | not null |
 email       | text            |           | not null |
 country     | text            |           | not null |
 updated_at  | timestamp without time zone |           | not null |
 source_system | text            |           |          |
Indexes:
    "users_replica_pkey" PRIMARY KEY, btree (id)

 id | name | email       | country | updated_at | source_system
---+---+---+---+---+---
  1 | ana  | ana@example.com | PT      | 2026-10-01 01:29:51.579 | users_db
  2 | raj  | raj@example.com | IN      | 2026-10-01 01:29:51.579 | users_db
  3 | li   | li@example.com  | CN      | 2026-10-01 01:29:51.579 | users_db
  4 | sam  | sam@example.com | US      | 2026-10-01 01:29:51.579 | users_db
(4 rows)

```

`auto.create=true` built the table **from the schema in the messages**: `int32` became `integer`, the `Timestamp` type became `timestamp`, and `pk.fields=id` became the primary key. The new column `source_system` came along, because to the sink it is just another field.

Exercise 5.5 · Changes flow through

Task: in `shop`, add a user and change li's country. What reaches the replica, and how?

✓ SOLUTION

PSQL

```

INSERT INTO users (name, email, country) VALUES ('mia', 'mia@example.com', 'DE');
UPDATE users SET country = 'SG' WHERE name = 'li';

```

A few seconds later, in `analytics`:

PSQL

```
SELECT id, name, country, updated_at, source_system FROM users_replica ORDER BY id;
```

OUTPUT

id	name	country	updated_at	source_system
1	ana	PT	2026-10-01 01:29:51.579	users_db
2	raj	IN	2026-10-01 01:29:51.579	users_db
3	li	SG	2026-10-01 01:38:43.729	users_db
4	sam	US	2026-10-01 01:29:51.579	users_db
5	mia	DE	2026-10-01 01:38:43.725	users_db

(5 rows)

The topic shows how: every `poll.interval.ms` (2 s), the source asks for rows whose `updated_at` or `id` is past its last offset.

OUTPUT

```
Partition:2 Offset:1 key=3 {"id":3,"name":"li","email":"li@example.com","country":"CN",...}
Partition:2 Offset:2 key=3 {"id":3,"name":"li","email":"li@example.com","country":"SG",...}
Partition:0 Offset:1 key=5 {"id":5,"name":"mia","email":"mia@example.com","country":"DE",...}
```

The update became a **new message** with the same key, in the same partition, after the old one. The topic keeps li's history. The sink, with `insert.mode=upsert`, turned it into an **UPDATE** of row 3.

The source's offset is again a position in **its** world, not Kafka's:

POWERSHELL

```
curl.exe -s localhost:8083/connectors/users-db-source/offsets
```

OUTPUT

```
{"offsets":[{"partition":{"protocol":"1","table":"public.users"},"offset":
{"timestamp_nanos":729000000,"incrementing":3,"timestamp":1790818723729}}]}
```

WHAT A POLLING SOURCE CANNOT SEE

A **deleted** row produces no message. The query cannot find a row that is no longer there. An **UPDATE** that does not change `updated_at` is missed too, which is why this table has a trigger. When you need deletes and every change, use log-based CDC: **Debezium**, which reads PostgreSQL's write-ahead log.

Lab 6 · Running Connect: Errors, Pausing, Scaling, Replaying

Goal: the jobs you do once connectors are running: handle a poison message with a **dead letter queue**, pause and resume a connector, add a second worker and lose it again, and replay a source from the beginning.

Exercise 6.1 · A poison message

Task: append a line that is not JSON to `orders.jsonl`. What happens to `orders-file-sink`, which writes text, and to `orders-clean-sink`, which parses JSON?

✓ SOLUTION

POWERSHELL

```
docker exec connect1 bash -c "echo this is not json >> /lab/data/in/orders.jsonl"
curl.exe -s localhost:8083/connectors/orders-file-sink/status
curl.exe -s localhost:8083/connectors/orders-clean-sink/status
```

OUTPUT

```
{
  "name": "orders-file-sink",
  "connector": {
    "state": "RUNNING",
    "tasks": [
      {
        "id": 0,
        "state": "RUNNING",
        "type": "sink"
      }
    ]
  },
  "type": "sink"
}
{
  "name": "orders-clean-sink",
  "connector": {
    "state": "RUNNING",
    "tasks": [
      {
        "id": 0,
        "state": "FAILED",
        "worker_id": "connect1:8083",
        "trace": "org.apache.kafka.connect.errors.ConnectException: Tolerance exceeded in error handler
          at org.apache.kafka.connect.runtime.errors.RetryWithToleranceOperator.execAndHandleError(...)
          ...
          Caused by: org.apache.kafka.connect.errors.DataException: Converting byte[] to Kafka Connect data failed due to
          serialization error:
          ...
          Caused by: com.fasterxml.jackson.core.JsonParseException: Unrecognized token 'this': was expecting (JSON String,
          Number, Array, Object or token 'null', 'true' or 'false')
          ..."}],
        "type": "sink"
      }
    ]
  },
  "type": "sink"
}
```

(Stack trace shortened.) The plain sink does not care: it wrote `this is not json` into its file. The clean sink's **task FAILED**, while its connector still says **RUNNING**, so always check the tasks. With the default `errors.tolerance=none`, a record that cannot be converted stops the task. It does not skip the record, because skipping silently would lose data. The bad record stays at its offset. Restarting the task alone would fail on it again.

Exercise 6.2 · A dead letter queue

Task: let `orders-clean-sink` skip records it cannot handle, but put them in a topic `shop-orders-dlq` so nothing is lost. Then look at what landed there.

✓ SOLUTION

`lab/connectors/orders-clean-sink-dlq.json` adds five lines to the previous config:

JSON

```
"errors.tolerance": "all",
"errors.deadletterqueue.topic.name": "shop-orders-dlq",
"errors.deadletterqueue.topic.replication.factor": "3",
"errors.deadletterqueue.context.headers.enable": "true",
"errors.log.enable": "true",
```

POWERSHELL

```
curl.exe -s -o NUL -w "HTTP %{http_code}\n" -X PUT localhost:8083/connectors/orders-clean-sink/config -H
"Content-Type: application/json" --data "@lab/connectors/orders-clean-sink-dlq.json"
curl.exe -s localhost:8083/connectors/orders-clean-sink/status
```

OUTPUT

```
HTTP 200
{"name":"orders-clean-sink","connector":{"state":"RUNNING",...},"tasks":
[{"id":0,"state":"RUNNING",...},"type":"sink"]
```

The config change restarted the failed task. This time the bad record went to the DLQ, and the task carried on:

KAFKA1 SHELL

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic shop-orders-dlq --from-beginning --max-
messages 1 \
--formatter-property print.headers=true
```

OUTPUT

```
__connect.errors.topic:shop-orders,__connect.errors.partition:1,__connect.errors.offset:0,
__connect.errors.connector.name:orders-clean-sink,__connect.errors.task.id:0,
__connect.errors.stage:VALUE_CONVERTER,__connect.errors.class.name:org.apache.kafka.connect.json.JsonConverter,
__connect.errors.exception.class.name:org.apache.kafka.connect.errors.DataException,
__connect.errors.exception.message:Converting byte[] to Kafka Connect data failed due to serialization error: ,
__connect.errors.exception.stacktrace:... this is not json
```

(Headers wrapped and the stack trace cut.) The original message, `this is not json`, is unchanged. The **headers** record where it came from (`shop-orders`, partition 1, offset 0), which **stage** failed (`VALUE_CONVERTER`) and why. Someone can fix it and produce it again, or a small consumer can do that automatically.

Dead letter queues exist only for **sink** connectors, and they catch failures in converters and transforms. An error thrown by the external system, for example a database constraint violation, is handled by the connector itself.

Exercise 6.3 · Pause and resume

Task: pause `orders-file-sink`, add an order, and look at the output file and the sink's lag. Then resume it.

✓ SOLUTION

POWERSHELL

```
curl.exe -s -i -X PUT localhost:8083/connectors/orders-file-sink/pause
curl.exe -s localhost:8083/connectors/orders-file-sink/status
docker exec connect1 bash /lab/add-order.sh 3011 frank "laptop stand" 55
Get-Content lab\data\out\shop-orders.txt -Tail 1
```

OUTPUT

```
HTTP/1.1 202 Accepted
{"name":"orders-file-sink","connector":{"state":"PAUSED",...},"tasks":[{"id":0,"state":"PAUSED",...},"type":"sink"]
{"order":3011,"customer":"frank","item":"laptop
stand","amount":55,"email":"frank@example.com","card":"4111111111111111"}
this is not json
```

KAFKA1 SHELL

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group connect-orders-file-sink
```

OUTPUT

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG	CONSUMER-ID
connect-orders-file-sink-0-019f231b	shop-orders	0	3	3	0	connector-consumer-orders-file-sink-0-019f231b
connect-orders-file-sink-0-019f231b	shop-orders	1	1	2	1	connector-consumer-orders-file-sink-0-019f231b

```
connect-orders-file-sink  shop-orders  2      8      8      0  connector-consumer-orders-file-sink-0-019f231b
```

(Columns trimmed as in Lab 3.) The order is in the topic, but not in the file: **lag 1**. A paused task stays in its group, so its partitions are not given to anyone else. It just stops processing.

```
curl.exe -s -i -X PUT localhost:8083/connectors/orders-file-sink/resume
Get-Content lab\data\out\shop-orders.txt -Tail 1
```

POWERSHELL

```
HTTP/1.1 202 Accepted
{"order":3011,"customer":"frank","item":"laptop
stand","amount":55,"email":"frank@example.com","card":"4111111111111111"}
```

OUTPUT

Pausing is how you hold back a sink while its target system is under maintenance. Nothing is lost: the messages wait in Kafka.

Exercise 6.4 · Scale out: a second worker

Task: start `connect2`, with the same `group.id` and a different host name. Where do the five connectors run now?

✓ SOLUTION

```
docker compose --profile scale up -d
curl.exe -s "localhost:8083/connectors?expand=status"
```

POWERSHELL

Shortened to one line per connector:

```
orders-clean-sink    connector@connect2:8083  tasks: 0 RUNNING@connect2:8083
orders-file-sink     connector@connect1:8083  tasks: 0 RUNNING@connect1:8083
orders-file-source   connector@connect1:8083  tasks: 0 RUNNING@connect1:8083
users-db-sink        connector@connect2:8083  tasks: 0 RUNNING@connect2:8083
users-db-source      connector@connect1:8083  tasks: 0 RUNNING@connect1:8083
```

OUTPUT

The new worker joined the Connect cluster `connect-lab`, and the group **rebalanced**: two connectors and their tasks moved to `connect2`. You did nothing but start a process. Either worker answers the REST API with the same picture, and a request to one worker for a connector on the other is forwarded. Both workers mount the same `plugins\` folder, so `connect2` can run the JDBC sink too. **Every worker needs every plugin.**

Exercise 6.5 · Losing a worker

Task: stop `connect2` and keep checking the status. How long until its connectors run again?

✓ SOLUTION

```
docker compose stop connect2
curl.exe -s "localhost:8083/connectors?expand=status"
```

POWERSHELL

```
right after the stop:
orders-clean-sink    connector@connect2:8083  tasks: 0 RUNNING@connect2:8083
users-db-sink        connector@connect2:8083  tasks: 0 RUNNING@connect2:8083
... the other three on connect1
```

OUTPUT

```

34 seconds later:
orders-clean-sink    connector@connect1:8083 tasks: 0 RUNNING@connect1:8083
orders-file-sink     connector@connect1:8083 tasks: 0 RUNNING@connect1:8083
orders-file-source   connector@connect1:8083 tasks: 0 RUNNING@connect1:8083
users-db-sink        connector@connect1:8083 tasks: 0 RUNNING@connect1:8083
users-db-source      connector@connect1:8083 tasks: 0 RUNNING@connect1:8083

```

Two lessons. First, **status can be stale**: a worker that is gone cannot report that its tasks stopped. Second, the surviving worker waited before taking over. Connect's rebalancing assumes a departed worker may be restarting, and gives it `scheduled.rebalance.max.delay.ms` to come back, so that a rolling restart does not move everything twice. The default is **5 minutes**. This lab sets 30 s, which is why the move took 34 s.

Exercise 6.6 · Replay a source

Task: make `users-db-source` read the whole table again. You must **stop** the connector, delete its offsets and resume it. Does the replica get duplicate rows?

✓ SOLUTION

```

curl.exe -s -i -X PUT localhost:8083/connectors/users-db-source/stop
curl.exe -s localhost:8083/connectors/users-db-source/offsets
curl.exe -s -X DELETE localhost:8083/connectors/users-db-source/offsets
curl.exe -s localhost:8083/connectors/users-db-source/offsets
curl.exe -s -i -X PUT localhost:8083/connectors/users-db-source/resume

```

OUTPUT

```

HTTP/1.1 204 No Content
{"offsets":[{"partition":{"protocol":"1","table":"public.users"},"offset":
{"timestamp_nanos":729000000,"incrementing":3,"timestamp":1790818723729}}]}
{"message":"The Connect framework-managed offsets for this connector have been reset successfully. However, if this
connector manages offsets externally, they will need to be manually reset in the system that the connector uses."}
{"offsets":[]}
HTTP/1.1 202 Accepted

```

```
kafka-get-offsets.sh --bootstrap-server kafka1:9092 --topic pg-users
```

PSQL

```
SELECT count(*) FROM users_replica;
```

OUTPUT

```

pg-users:0:4
pg-users:1:2
pg-users:2:5
count
-----
      5

```

STOPPED is stronger than **PAUSED**: the tasks are shut down completely, which is what Connect requires before it lets you change offsets. With the offsets gone, the source read all 5 rows again: the topic grew from 6 to **11** messages. The replica still has **5 rows**. The sink **upserts** by primary key, so writing the same row twice has the same effect as writing it once. This is the at-least-once world of Kafka: **make the sink idempotent**, and duplicates do no harm.

Lab 7 · Cheat Sheet and Troubleshooting

The REST API

Base URL `http://localhost:8083`. Every worker answers for the whole Connect cluster.

Call	What it does
<code>GET /</code>	Worker version and the Kafka cluster id
<code>GET /connector-plugins[?connectorsOnly=false]</code>	Connector plugins the worker can load (plus transforms, predicates, converters)
<code>GET /connectors[?expand=status&expand=info]</code>	All connectors, optionally with status and config
<code>PUT /connectors/{name}/config</code>	Create or update a connector. Body: the config JSON. <code>201</code> new, <code>200</code> updated
<code>GET /connectors/{name}/status</code>	Connector and task states, worker ids, the stack trace of a failed task
<code>GET /connectors/{name}/config</code>	The current config
<code>PUT /connectors/{name}/pause</code> · <code>/resume</code>	Hold processing. Tasks stay assigned
<code>PUT /connectors/{name}/stop</code>	Shut the tasks down completely: state <code>STOPPED</code>
<code>POST /connectors/{name}/restart?includeTasks=true&onlyFailed=true</code>	Restart the connector and (only the failed) tasks
<code>GET /connectors/{name}/offsets</code>	Source: positions in the source system. Sink: consumer offsets
<code>DELETE /connectors/{name}/offsets</code>	Reset offsets. The connector must be <code>STOPPED</code>
<code>DELETE /connectors/{name}</code>	Remove the connector. Its topics and committed offsets stay

With `curl.exe` in PowerShell: send config bodies from a file with `--data "@file.json"`, and put URLs that contain `?` or `&` in quotes.

Connector settings used in this lab

Setting	Where	Meaning
<code>connector.class</code> , <code>tasks.max</code>	all	The plugin, and the upper limit of parallel tasks
<code>topic</code> / <code>topics</code> , <code>topics.regex</code>	source / sink	Where to write / what to read
<code>key.converter</code> , <code>value.converter</code> , <code>...converter.schemas.enable</code>	all	Override the worker's converters
<code>topic.creation.default.partitions</code> , <code>.replication.factor</code>	source	Let Connect create the topics it writes to
<code>transforms</code> , <code>transforms.<name>.type</code> , <code>transforms.<name>.*</code>	all	The SMT chain, in order
<code>predicates</code> , <code>predicates.<name>.type</code> , <code>transforms.<t>.predicate</code>	all	Apply a transform only to matching records

Setting	Where	Meaning
<code>errors.tolerance</code> , <code>errors.deadletterqueue.*</code> , <code>errors.log.enable</code>	sink	Skip bad records and keep them in a DLQ
<code>mode</code> , <code>incrementing.column.name</code> , <code>timestamp.column.name</code> , <code>topic.prefix</code> , <code>poll.interval.ms</code>	JDBC source	How changed rows are found
<code>insert.mode</code> , <code>pk.mode</code> , <code>pk.fields</code> , <code>auto.create</code> , <code>table.name.format</code>	JDBC sink	How rows are written

Built-in SMTs at a glance

SMT	Does
<code>InsertField</code>	Adds a fixed value, or the topic, partition, offset or timestamp
<code>ReplaceField</code>	<code>include</code> / <code>exclude</code> fields, <code>renames=old:new</code>
<code>MaskField</code>	Replaces field values with a null-like value or a <code>replacement</code>
<code>ValueToKey</code> , <code>ExtractField</code>	Build the key from value fields; pull one field out of a struct
<code>Cast</code> , <code>TimestampConverter</code>	Change a field's type or a timestamp's format
<code>HoistField</code> , <code>Flatten</code>	Wrap a value in a struct; flatten nested structs to <code>a.b</code> names
<code>RegexRouter</code> , <code>TimestampRouter</code>	Change the topic name
<code>Filter</code> + <code>HasHeaderKey</code> / <code>TopicNameMatches</code> / <code>RecordIsTombstone</code>	Drop matching records
<code>InsertHeader</code> , <code>HeaderFrom</code> , <code>DropHeaders</code>	Work with headers

Troubleshooting

Symptom	Cause	Fix
<code>Add-Content : ... being used by another process</code>	The FileStream source holds the file open, and Docker Desktop then blocks writes from Windows	Append inside the container: <code>docker exec connect1 bash /lab/add-order.sh ...</code>
<code>Failed to find any class that implements Connector ... JdbcSourceConnector</code>	The plugin is not on <code>plugin.path</code> , or the worker started before it was installed	Check <code>plugins\</code> , then <code>docker compose restart connect1</code> (and <code>connect2</code>)
Source task FAILED: <code>UNKNOWN_TOPIC_OR_PARTITION</code> / topic not present	The cluster has <code>auto.create.topics.enable=false</code>	Add <code>topic.creation.default.replication.factor</code> and <code>.partitions</code> , or create the topic first
Sink task FAILED: <code>JsonParseException</code>	A message is not JSON, but the converter is <code>JsonConverter</code>	Fix the producer, or use <code>errors.tolerance=all</code> with a DLQ (Lab 6)
<code>JsonConverter</code> with <code>schemas.enable</code> requires "schema" and "payload" fields	The sink expects the schema envelope, but the topic has plain JSON	Make producer and sink agree on <code>schemas.enable</code> - or use Schema Registry

Symptom	Cause	Fix
JDBC sink: <code>Sink connector ... requires records with a non-null Struct value and non-null Struct schema</code>	The data has no schema (plain JSON)	<code>value.converter.schemas.enable=true</code> on both sides, or Avro with Schema Registry
Messages show up as <code>{"order\":"...</code>	A string was written by <code>JsonConverter</code>	Use <code>StringConverter</code> for text that is already JSON (Lab 2.5)
Status says RUNNING for a worker that is gone	A dead worker cannot update its status	Wait for the rebalance - up to <code>scheduled.rebalance.max.delay.ms</code> (5 min by default)
A connector's tasks do nothing after a worker restart	Same delay, while the cluster waits for the worker to return	Same - or restart the connector

Cleaning up

POWERSHELL

```
# delete one connector (its topics and data stay)
curl.exe -s -X DELETE localhost:8083/connectors/orders-clean-sink

# stop the Lab, keep the PostgreSQL data / and delete it
docker compose --profile scale down
docker compose --profile scale down -v
```

KAFKA1 SHELL

```
# the topics this Lab created, if you want them gone
kafka-topics.sh --bootstrap-server kafka1:9092 --delete --topic 'shop-orders.*|pg-users'
# the Connect cluster's state - only with every worker stopped
kafka-topics.sh --bootstrap-server kafka1:9092 --delete --topic 'connect-(configs|offsets|status)'
```

Deleting the three `connect-*` topics resets the Connect cluster completely: every connector config and every source offset is gone.